

NOPT: A Learning-Enhanced Metaheuristic Solver for Combinatorial Optimization Problems

Lei Li^a, Yonghao Du^{a,*}, Feng Yao^a, Yingguo Chen^a, Lining Xing^{b,c}, Yuyang Lai^{d,e}, Guohua Wu^f and Bin Xin^g

^aCollege of Systems Engineering, National University of Defense Technology, 410073, Changsha, China

^bKey Laboratory of Collaborative Intelligence Systems, Ministry of Education, Xidian University, 710071 Xi'an, China

^cResearch Institute of Intelligent Control and Manufacturing System, Jiangsu University of Technology, 213001 Changzhou, China

^dDepartment of Automation, Tsinghua University, 100084 Beijing, China

^eBeijing Shuyou Information Technology Co., Ltd., 100176 Beijing, China

^fSchool of Automation, Central South University, 410083 Changsha, China

^gSchool of Automation, Beijing Institute of Technology, 100081 Changsha, China

ARTICLE INFO

Keywords:

COPs

Metaheuristic solver

Learning mechanism

Modeling and solving

ABSTRACT

Combinatorial optimization problems (COPs) arise widely in resource allocation, scheduling, and planning, yet existing solvers and metaheuristic algorithms remain limited by strong expert dependency, high customization costs, and poor scalability for complex engineering COPs. This paper presents a nature-inspired optimization (NOPT) solver, enhanced by learning mechanisms, which enables unified modeling and efficient solving of diverse COPs. First, we introduce a generalized modeling method based on *decision instances*, a fundamental data structure that abstracts decision relationships into standardized units, along with a hard-constraint–objective–soft-constraint evaluation strategy for consistent assessment of heterogeneous COPs. Second, we design a Learning Metaheuristic Algorithm (LMeHA) framework that accommodates both neighborhood search and evolutionary algorithms through interconvertible decision instance encoding and decision matrix encoding. An reinforcement learning mechanism with a late acceptance reward strategy is incorporated for adaptive operator selection across different optimization phases. Extensive experiments on 10 classical COPs, including TSP, VRP, and JSP, and 8 engineering COPs, including satellite scheduling and gate assignment, are conducted against CPLEX, OR-Tools, and multiple metaheuristics. Experiment results show that LMeHA achieves the best average ranking of 1.79, outperforming conventional metaheuristics, while NOPT demonstrates broader constraint expressiveness and superior solving performance on complex engineering COPs intractable to exact solvers, thereby confirming strong cross-scenario adaptability for industrial applications.

1. Introduction

Combinatorial optimization problems (COPs) [1] arise extensively in aerospace [2], intelligent manufacturing [3], defense [4], and other domains. Conventional approaches to solving COPs typically exhibit strong problem dependency, requiring repeated problem formulation, constraint modeling, and algorithm design for different application scenarios, which leads to complex solution workflows and heavy reliance on domain expertise [5, 6]. Although customized methods can achieve satisfactory performance on specific problems, developing a generalized modeling and solving framework for efficient resolution of complex industrial COPs remains a critical research direction in operations research.

The inherent properties of COPs and the complexity of industrial scenarios pose substantial challenges in constraint modeling and solution processes. The vast majority of classical COPs have been proven NP-hard [1], exhibiting a “combinatorial explosion” phenomenon in which the

solution space and computational complexity grow exponentially or factorially with problem size [7]. From an engineering perspective, complex industrial COPs such as satellite scheduling [8, 9] and human resource scheduling [10] are typically characterized by tightly coupled multi-constraints, enormous decision spaces, and non-convex, nonlinear constraint-objective structures, further increasing the solving difficulty. Moreover, the constraints and objectives of COPs vary significantly across scenarios [11], and even similar scenarios often defy unified modeling and solving due to problem-specific structures.

To address these challenges, researchers have developed a diverse set of methods, including exact algorithms, metaheuristic algorithms, and machine learning-enhanced approaches, each exhibiting distinct trade-offs in computational efficiency, applicability, and optimization mechanisms. Exact algorithms [12, 13, 14] offer theoretical guarantees of solution optimality but are highly sensitive to problem scale and constraint complexity, making them difficult to apply directly in large-scale, strongly constrained industrial scenarios [15]. Metaheuristic algorithms [16, 17, 18] iteratively improve candidate solutions guided by objective functions [19, 20] and can obtain satisfactory solutions within acceptable timeframes [21]. Based on their search

*Corresponding author

✉ lilei19@nudt.edu.cn (L. Li); duyonghao15@163.com (Y. Du);

yaofeng@nudt.edu.cn (F. Yao); argguo@163.com (Y. Chen);

lnxing@xidian.edu.cn (L. Xing); 16376313@qq.com (Y. Lai);

guohuawu@csu.edu.cn (G. Wu); brucebin@bit.edu.cn (B. Xin)

Table 1
Typical existing solvers for COPs.

Solver	Developer	Year	Characteristics
CPLEX	IBM	1988	Commercial; LP, MIP, SOCP; exact algorithms
Gurobi	Gurobi Opt.	2009	Commercial; MIP, MIQCP; exact algorithms
COPT	Cardinal Opt.	2019	Commercial; LP, MIP; exact algorithms, GPU-accelerated
SCIP	ZIB	2002	Academic open-source; constraint integer programming; exact and heuristic algorithms
MindOpt	Alibaba	2020	Open-source; LP, MIP, MIQCP; exact algorithms with parameter tuner
OR-Tools	Google	2010	Open-source; routing, packing, scheduling; exact (CP-SAT) and heuristic engines
Timefold	Red Hat	2006	Open-source; entity-based planning; heuristic algorithms

mechanisms, metaheuristics can be categorized into neighborhood search algorithms and evolutionary algorithms. Neighborhood search algorithms include Hill Climbing Algorithm[22], Late Acceptance Algorithm[23], Simulated Annealing Algorithm[24], Tabu Search Algorithm[25], and Iterated Local Search Algorithm[26], which perform fine-grained local exploitation through designed neighborhood structures and acceptance criteria, typically achieving fast convergence. Evolutionary algorithms include Particle Swarm Optimization[27, 28], Genetic Algorithms [29, 30] and Memetic Algorithms [31, 32], which simulate biological evolution and collective behavior, employing selection, crossover, and other evolutionary operators to conduct global exploration while maintaining strong population diversity. With the advancement of artificial intelligence, machine learning has provided new technical pathways for solving COPs. Learning-enhanced optimization algorithms integrate machine learning techniques into metaheuristic frameworks [31, 33], leveraging historical experience for search strategy guidance and dynamic parameter control [34, 35], thereby improving algorithmic intelligence. However, such learning mechanisms require scenario-specific design of fine-grained algorithm modules and customized features. End-to-end methods construct neural networks or reinforcement learning models [36, 37] to directly learn mappings from problem inputs to feasible solutions, yet their reliance on high-quality training data limits cross-scenario generalization and complex constraint handling capabilities.

General-purpose solvers, capable of accommodating standardized problem models and providing unified solving capabilities, have become essential infrastructure for industrial optimization. Commercial solvers such as CPLEX [38], GUROBI [39], COPT [40], SCIP[41], and MindOpt[42], built upon mature mathematical programming methods, can obtain theoretically optimal solutions for small-to-medium-scale COPs with regular constraint structures. OR-Tools

[43] encapsulates multiple modular engines including CP-SAT, MIP and Routing, providing specialized optimization strategies for certain classical COPs such as scheduling, sequencing, and assignment. However, due to its highly encapsulated model structure, OR-Tools struggles with flexible cross-domain extension when problem scenarios change or constraints become intractable. Furthermore, both commercial solvers and open-source optimization tools have limited expressive power for non-standard optimization problems and cannot handle COPs with implicit or non-analytic constraints and objectives. Open-source solution frameworks such as OptaPlanner [44], which is the predecessor of Timefold, provide modular support for heuristic algorithms, yet they typically lack unified modeling mechanisms, requiring problem-specific design of solution encoding, search operators, and solving workflows, which results in high development costs. Moreover, these frameworks remain dominated by traditional metaheuristic algorithms [45], whose optimization performance depends heavily on manual parameter tuning, with only preliminary integration of machine learning and other advanced techniques. Table 1 lists these representative solvers with their development timeline and core features.

In summary, existing COP solvers and algorithm frameworks exhibit three key limitations: constrained solving capability under large-scale, complex scenarios, and limited modeling flexibility due to strict formalization requirements and insufficient constraint expressiveness. Moreover, methodological generality remains inadequate, as metaheuristic approaches require problem-specific customized design, rely on expert experience, and lack adaptive optimization mechanisms for cross-scenario transferability.

To address these issues, this paper proposes a standardized COP modeling method based on decision instances and a metaheuristic algorithm framework enhanced by reinforcement learning. Based on these, we develop NOPT, a general-purpose solver that integrates both approaches. The main contributions are as follows:

- A generalized modeling method for COPs is proposed by defining the fundamental data structure of *decision instance*. Decision objects across different scenarios are standardized as *decision instances* that assign values to decision variables. A “hard constraint–objective value–soft constraint” evaluation strategy is constructed, enabling unified modeling and quantitative assessment of both classical and complex-constrained COPs.
- A learning-enhanced metaheuristic algorithm framework is designed that accommodates both neighborhood search and evolutionary algorithms through interconvertible decision instance encoding and decision matrix encoding. A generalized operator pool containing move, swap, and other universal operations is established. And a reinforcement learning strategy is incorporated to accumulate optimization knowledge during the iterative process for guiding subsequent

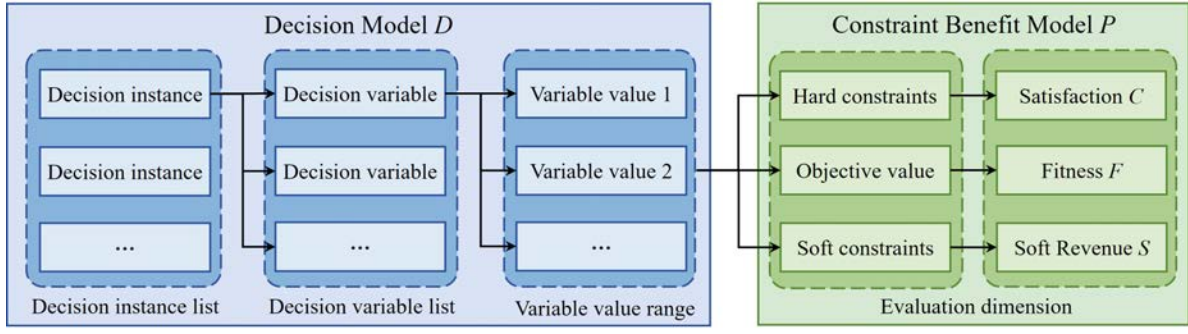


Figure 1: Relationship between the decision model and the constraint-benefit model in the generalized COP description.

search, thereby enhancing adaptive optimization capability across different search phases and scenarios.

- The proposed framework is developed in native Java with fully open-source code, facilitating engineering deployment and reproducibility. Extensive experiments on 10 classical COPs and 8 engineering COPs against CPLEX, OR-Tools, and multiple metaheuristics thoroughly validate the generality and effectiveness of the proposed framework.

The remainder of this paper is organized as follows. Section 2 presents the generalized modeling method based on decision instances. Section 3 details the learning-enhanced metaheuristic optimization framework and algorithm. Section 4 reports the experimental evaluation. Section 5 concludes this paper with future research directions.

2. Decision Instance-Based Modeling

This section proposes a generalized modeling method based on decision instances, building upon a standardized COP model. The method achieves a universal description of diverse COPs while constructing a decision model and a constraint-benefit evaluation model, providing a standardized model foundation and evaluation basis for the subsequent design of general-purpose solving algorithms.

2.1. Generalized Problem Description

A standardized and generalized representation of various COPs is first established by defining the concept of a decision instance:

Definition 1 (Decision Instance): In the formulation of a combinatorial optimization problem, a decision instance is a fundamental data structure represented in an instantiated form, embedding one or more instantiated decision variables along with their respective value ranges.

The standardized mathematical model of a COP is given as follows [46], where x is a d -dimensional discrete decision vector, $f(\cdot)$ is the optimization objective function, $g_i(\cdot)$ and $h_j(\cdot)$ denote inequality and equality constraints respectively,

and Ω represents the discrete feasible decision space:

$$\begin{aligned} \min \quad & f(x) \\ \text{s.t.} \quad & g_i(x) \leq 0, \quad i = 1, 2, \dots, p \\ & h_j(x) = 0, \quad j = 1, 2, \dots, q \\ & x \in \Omega \end{aligned} \quad (1)$$

Any COP can be described using decision instances as its minimal units, with binary 0-1 variables capturing the selection state of each instance. As illustrated in Fig. 1, a decision instance may contain multiple decision variables, each taking values within a specified range. In a COP, every feasible solution corresponds uniquely to a combination of decision instances. Let the k -th decision instance be I_k with an associated p -dimensional binary variable y_k ; then there exists a bijection $\theta : \Omega \rightarrow \{0, 1\}^p$ that transforms the original problem into:

$$\begin{aligned} \min \quad & f(\theta^{-1}(y_1, y_2, \dots, y_m)) \\ \text{s.t.} \quad & g_i(\theta^{-1}(y_1, y_2, \dots, y_m)) \leq 0, \quad i = 1, 2, \dots, p \\ & h_j(\theta^{-1}(y_1, y_2, \dots, y_m)) = 0, \quad j = 1, 2, \dots, q \\ & y_k \in \{0, 1\}, \quad k = 1, 2, \dots, m \end{aligned} \quad (2)$$

where the constraints and objective function maintain equivalence through the inverse mapping.

A COP can thus be described by a decision model D and a constraint-benefit evaluation model P , i.e., $\text{COP} = \{D, P\}$. As shown in Fig. 1, the decision model D decouples the problem model from the solving algorithm by describing the decision elements and solution space, and consists of decision instances E , decision variables Y , and value ranges V , i.e., $D = \{E, Y, V\}$. The evaluation model P quantifies solution legality, optimization effectiveness, and engineering applicability, comprising hard constraints C , optimization objectives F , and soft benefits S , i.e., $P = \{C, F, S\}$. Given that hard constraints C and optimization objectives F are satisfied, the soft benefits S quantify non-mandatory preference requirements to enhance solution executability.

This modeling approach abstracts scenario-specific business details through decision instances and relies on the constraint-objective-benefit evaluation mechanism, covering classical COPs such as routing and resource allocation

Table 2

Specific Examples of Decision Models for Different Combinatorial Optimization Problems (COPs) Based on Decision Instances

	Problem Name	Decision Instance	Decision Variable	Variable Type
	Assignment Problem (AP)	Task	Resource	Instantiated object
	Bin Packing Problem (BPP)	Item	Packing priority Rotation flag	Positive integer Boolean
	Graph Coloring Problem (GCP)	Node	Color	Instantiated object
	Job Shop Problem (JSP)	Operation	Machine Operation priority	Instantiated object Positive integer
	Knapsack Problem (KP)	Item	Knapsack	Instantiated object
	N-Queens Problem (N-QP)	Queen	Grid cell	Instantiated object
	Orienteering Problem with Time Windows (OPTW)	Vertex	Route Route priority	Instantiated object Positive integer
	Function Optimization (FO)	Variable	Discrete value	Floating-point number
	Traveling Salesman Problem (TSP)	City	Priority	Positive integer
	Capacitated Vehicle Routing Problem with Time Windows (CVRPTW)	Customer	Vehicle Access priority	Instantiated object Positive integer
	Earth Observation Satellite Scheduling Problem (EOSSP)	Task	Imaging time window Extension rate	Instantiated object Integer
	Satellite Antenna Scheduling Problem (SASP)	Task	Time window	Instantiated object
	Launch Site Scheduling Problem (LSSP)	Task	Launch pad Priority	Instantiated object Positive integer
	Network Disintegration Problem (NDP)	Node Edge	Disintegration flag Disintegration flag	Boolean Boolean
	Human Resource Scheduling Problem (HRSP)	Person	Transferable position	Instantiated object
	Navigation Satellite Scheduling Problem (NSSP)	Task	Link establishment flag	Boolean
	Multi-platform Path Planning Problem (MPP)	Platform	Target point	Instantiated object
	Airport Gate Assignment Problem (AGAP)	Plane	Gate	Instantiated object

while accommodating the individualized optimization requirements of complex industrial scenarios such as earth observation satellite scheduling and space launch scheduling.

2.2. Decision Model Based on Decision Instances

The core components and semantics of the decision model D are elaborated below. Table 2 presents specific instances of decision model construction for various COPs based on decision instances, using 10 types of classical problems and 8 types of engineering problems as examples. The decision instances and variable implementation for each problem in NOPT can be found in Appendix A.

- Decision Instances:** As the fundamental data structure in COP descriptions, decision instances transform the decision objects in a given scenario into standardized selectable units. For example, operations to be processed in JSP, customers to be visited in CVRPTW, and imaging tasks to be executed in EOSSP can all be abstracted as decision instances. The abstraction is independent of specific business scenarios and should be constructed according to the essential decision-making nature of the problem.
- Decision Variables:** Decision variables connect decision instances with optimization objectives and constraints, representing inherent attributes of decision entities, relationships between entities, or selection logic for specific attributes. For example, the processing priority and target machine for operations in JSP, customer priority in CVRPTW, and the imaging windows with extension rate for tasks in EOSSP all serve as decision variables. A single decision instance may embed multiple types of decision variables to satisfy the modeling and solving requirements of complex problem scenarios.
- Variable Ranges:** Value ranges define the feasible domain boundaries of decision variables, constituting the discrete set of all possible values a decision variable may take. Examples include a processing priority range of $\{1, 2, \dots, 10\}$, a binary selection variable $\{0, 1\}$, and a set of selectable resource instances $\{\text{Resource A}, \text{Resource B}, \text{Resource C}\}$. Multiple types of discrete sets are supported, including discrete, integer, instance-based resource, and null types.

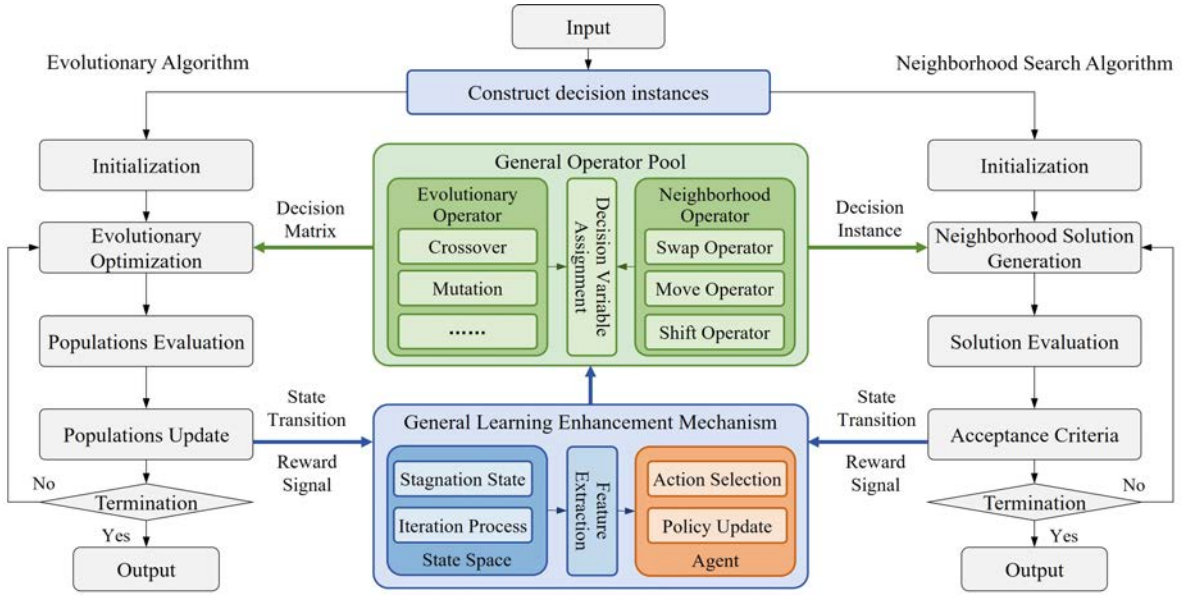


Figure 2: Framework of the Learning-Enhanced Metaheuristic Algorithm.

3. Learning Metaheuristic Algorithm

This section presents the LMeHA algorithm that uses decision instances as the basic operational units, designs generalized neighborhood operators, and constructs a reinforcement learning-enhanced adaptive operator selection mechanism, enabling efficient and intelligent optimization of various COP models.

3.1. Algorithm Framework

A solving framework that accommodates both neighborhood search and evolutionary algorithms is designed, incorporating a learning enhancement mechanism and two types of encoding structures, providing NOPT with a general-purpose algorithmic architecture. The algorithm framework is illustrated in Fig. 2.

3.1.1. Neighborhood Search Algorithms

Neighborhood search algorithms take single or small-scale candidate solutions as the iterative basis, performing local exploitation through neighborhood structure definitions and updating the current solution according to predefined acceptance criteria. Their optimization performance is limited by the adaptability of neighborhood structures and the self-adaptiveness of acceptance criteria.

To overcome the reliance on problem-specific neighborhood operators and manual parameter tuning in traditional neighborhood search algorithms, NOPT standardizes problem encoding based on decision instances and abstracts the neighborhood transformation operations of different COPs into universal adjustment rules on encoding sequences, enabling cross-problem reuse of neighborhood structures. Meanwhile, the embedded learning enhancement mechanism adaptively optimizes neighborhood structure selection strategies and key parameters of acceptance criteria, replacing manually preset fixed rules and improving the

Algorithm 1 Learning Neighborhood Search in NOPT

```

1: Input: Decision instance list  $I$ , operator pool  $A$ , max iterations  $T_{\max}$ , late acceptance list size  $L$ , learning rate  $\alpha$ , discount factor  $\gamma$ , exploration rate  $\epsilon$ 
2: Output: Best solution  $Y_{\text{best}}$ 
3:  $Y \leftarrow \text{Init}(I)$  via Algorithm 3;  $Y_{\text{best}} \leftarrow Y$ ;  $Q \leftarrow \mathbf{0}$ ;  $\mathcal{L} \leftarrow \{F(Y)\}$ 
4: for  $t = 1$  to  $T_{\max}$  do
5:    $s_t \leftarrow \{\rho, \omega\}$ 
6:    $a_t \leftarrow \epsilon\text{-greedy}(Q, s_t)$ 
7:   Apply operator  $a_t$  to  $Y$ ; generate  $Y_{\text{new}}$ 
8:    $R(s_t, a_t) \leftarrow \text{Eq. (5)}$ 
9:   Update  $Q$  via Eq. (7)
10:  if  $F(Y_{\text{new}}) > F(Y_{\text{best}})$  then
11:     $Y_{\text{best}} \leftarrow Y_{\text{new}}$ 
12:  end if
13:  Accept or reject  $Y_{\text{new}}$  based on acceptance criteria
14:   $\mathcal{L}.\text{enqueue}(F(Y)); \mathcal{L}.\text{dequeue}()$  if  $|\mathcal{L}| > L$ 
15: end for
16: return  $Y_{\text{best}}$ 

```

algorithm's ability to escape local optima and its iterative optimization efficiency. The overall procedure of the learning-enhanced neighborhood search algorithm is summarized in Algorithm 1.

3.1.2. Evolutionary Algorithms

Evolutionary algorithms achieve global exploration of the solution space through population-based iterative strategies, employing evaluation, selection, crossover, mutation, and update operations to propagate and recombine promising solution structures.

In NOPT, evolutionary algorithms are based on the three-dimensional continuous representation matrix designed

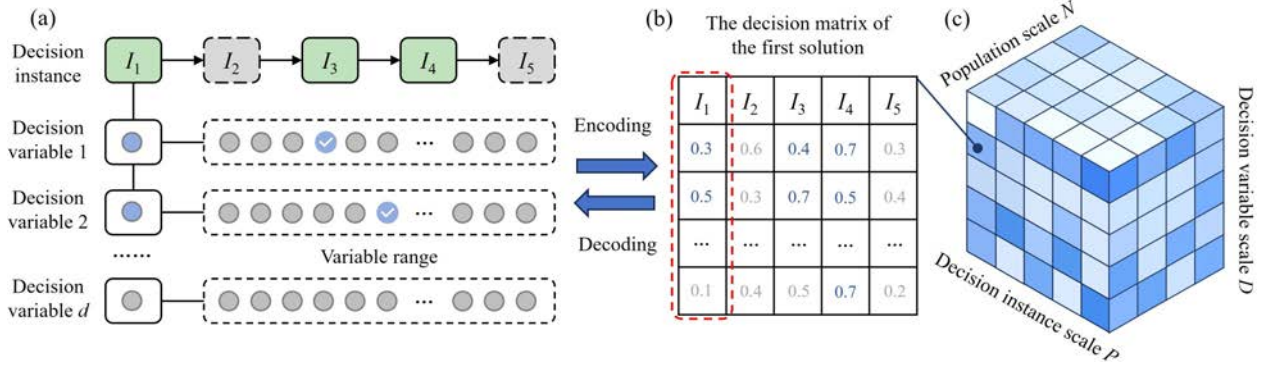


Figure 3: Solution encoding methods: (a) Decision instance encoding; (b) Single-individual decision matrix encoding; (c) Population decision matrix encoding.

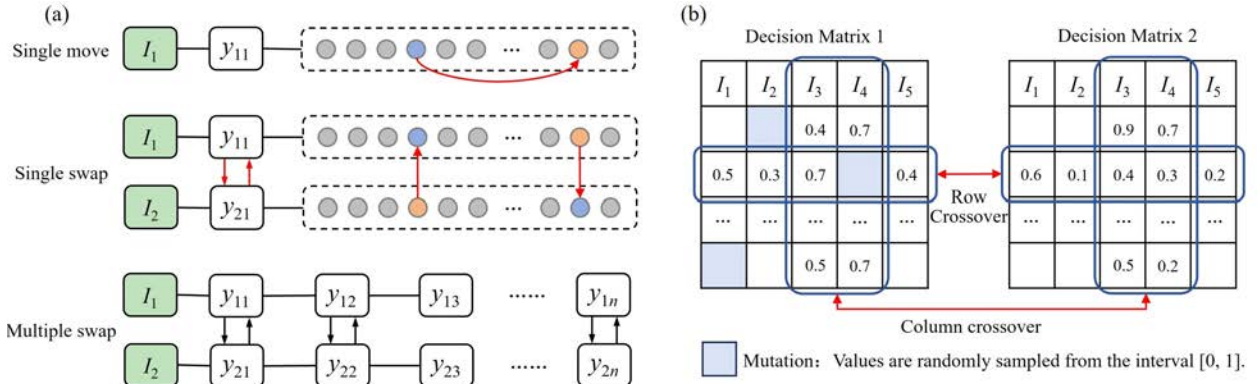


Figure 4: Illustration of operators: (a) Neighborhood search operators; (b) Evolutionary algorithm operators.

in Section 3.2, which maps traditional discrete-space evolutionary operators into standardized transformations in a continuous matrix space. Moreover, the population and matrix features serve as input for the embedded learning enhancement mechanism, enabling data-driven learning to adaptively adjust population evolution strategies and balance global exploration with local exploitation. The overall procedure of the learning-enhanced evolutionary algorithm is summarized in Algorithm 2.

3.2. Solution Encoding And Decoding

Generalized encoding methods adapted to both neighborhood search and evolutionary algorithms are designed as shown in Fig. 3. For neighborhood search algorithms, decision instance encoding serves directly as the solution representation; based on the selection results of decision variables within each decision instance, all selected decision instances can be assigned or arranged according to preset rules, as illustrated in Fig. 3(a).

For evolutionary algorithms, a three-dimensional representation matrix is constructed based on decision instances, mapping original discrete decision variables into a continuous interval. Let the set of decision instances be $I = \{I_1, I_2, \dots, I_p\}$ with p being the total number of decision instances. Each decision instance I_k has an associated set of decision variables $Y_k = \{y_{k1}, y_{k2}, \dots, y_{kn}\}$ with n denoting

Algorithm 2 Learning Evolutionary Algorithm in NOPT

- 1: **Input:** Decision instance list I , operator pool A , population size N , max iterations $T_{\max}$, crossover probability p_c , mutation probability p_m , late acceptance list size L , learning rate α , discount factor γ , exploration rate ϵ
- 2: **Output:** Y_{best}
- 3: $\mathcal{P} \leftarrow \text{Init}(I, N)$ via Algorithm 3; $\mathcal{Q} \leftarrow \mathbf{0}$; $\mathcal{L} \leftarrow \{F(Y_{\text{best}})\}$
- 4: **for** $t = 1$ **to** $T_{\max}$ **do**
- 5: $s_t \leftarrow \{\rho, \omega\}$; $a_t \leftarrow \epsilon\text{-greedy}(\mathcal{Q}, s_t)$
- 6: Select parents; apply crossover and mutation on $\mathbf{M}$
- 7: Decode offspring via Eq. (4) and evaluate fitness
- 8: $R(s_t, a_t) \leftarrow \text{Eq. (5)}$; update $\mathcal{Q}$ via Eq. (7)
- 9: $\mathcal{P} \leftarrow \text{Update}(\mathcal{P}, \text{offspring})$
- 10: $Y_{\text{best}} \leftarrow \arg \max_{Y \in \mathcal{P}} F(Y)$
- 11: **end for**
- 12: **return** Y_{best}

the number of decision variables for I_k . The value range of decision variable y_{kt} is $V_{kt} = \{v_{kt1}, v_{kt2}, \dots, v_{ktr}\}$ with r being the number of possible values. Given a population size of N , the decision matrix $\mathbf{M} \in [0, 1]^{N \times p \times n}$ is constructed. For the i -th solution in the population, the k -th decision instance, and the t -th decision variable, if the selected value in the solution is v_{ktm} with $1 \leq m \leq |V_{kt}|$, then the encoding value

at the corresponding matrix position is given by Eq. (3). During decoding, the encoding value is multiplied by the length of the decision variable's value range and floored to recover the original discrete value index, as in Eq. (4):

$$\mathbf{M}_{i,k,t} = \frac{m - 0.5}{|V_{kt}|} \quad (3)$$

$$m = \lfloor \mathbf{M}_{i,k,t} \times |V_{kt}| \rfloor + 1 \quad (4)$$

where $\lfloor \cdot \rfloor$ denotes the floor function. Through index m , the original decision variable value v_{ktm} can be directly recovered from the value range V_{kt} .

3.3. General Random Sampling Initialization

A random sampling initialization strategy is designed to provide diverse initial solutions for both types of algorithms. As described in Algorithm 3, instance encoding initialization randomly assigns values within the value range to decision variables and updates the decision instance states, whereas decision matrix initialization generates initial solutions through random filling of matrix elements, which can then be mapped to decision instance encoding via the encoding conversion in Section 3.2 for fitness evaluation.

Algorithm 3 Random Sampling Initialization

```

1: Input: Decision instance list  $I$ , population size  $N$ , strategy
2: Output: Initialized solution(s)
3: if strategy is instance encoding then
4:    $Y \leftarrow \text{RandomSelect}(V)$  for all instances in  $I$ 
5:   UpdateScore( $Y, I$ )
6: else
7:   for  $n = 1$  to  $N$  do
8:      $\mathbf{M}_n \leftarrow \text{Encode}(I)$ 
9:      $\mathbf{M}_n \sim \mathcal{U}(0, 1)$  ▷ Encode & randomize
10:    Decode( $\mathbf{M}_n, I$ ) via Eq. (4)
11:    Evaluate( $I$ )
12:   end for
13: end if

```

3.4. Solution Operators

Based on the two encoding structures, this section designs multiple types of generalized neighborhood operators that select decision instances through random, greedy, or other rules and apply neighborhood perturbation operations to their decision variables, thereby accomplishing solution transformations. Several representative neighborhood operators are illustrated in Fig. 4.

- **Swap:** Selects two decision instances and exchanges the assigned values of one or more of their decision variables.
- **Move:** Randomly selects a value different from the current assignment within the decision variable's value range and reassigns it.

- **Shift:** Selects the next or an interval-shifted value within the value range according to a predefined order and reassigns it.
- **Mutation:** Randomly alters the $[0, 1)$ interval value at a specified position within a single solution's decision matrix.
- **Crossover:** Selects the decision matrices of two solutions and exchanges values at specified row and column positions.

These operators primarily rely on random selection of decision instances and decision variables during the solving process. They can also be extended to incorporate problem-specific heuristic information for selection, or combined within a single iteration to explore a broader neighborhood space, thereby enhancing optimization performance on particular problems.

3.5. Learning Adaptive Operator Selection

During the optimization process, the selection of neighborhood operators directly determines search efficiency and optimization direction. Traditional methods predominantly employ fixed sequences or handcrafted rules, which may lead to insufficient optimization adaptivity. Online reinforcement learning is employed to dynamically update operator selection strategies based on feature states of the iterative process, thereby improving adaptive operator selection [47]. The core elements are as follows:

1) State space S : The state is defined as a set of environmental features of the search process, $s = \{\rho, \omega\}$, where ρ denotes the number of consecutive iterations trapped in a local optimum, discretized into three levels to reflect the stagnation status, and ω denotes the iteration progress, discretized into two levels to enable dynamic adjustment across optimization phases.

2) Action space A : The action set $A = \{a_1, a_2, \dots, a_m\}$ corresponds one-to-one with the candidate neighborhood operator list, which remains fixed during a single algorithm run. In each iteration, an action is selected based on the current state using an ϵ -greedy rule, balancing exploitation of learned knowledge with random exploration.

3) Reward function R : The reward function is critical for guiding strategy selection. Traditional strategies typically focus on immediate benefit improvement, leading to myopic optimization and sparse rewards that make the algorithm prone to local optima. A late acceptance reward strategy is designed to encourage exploration within a benefit threshold, as defined in Eq. (5):

$$R(s, a) = \begin{cases} r^+, & \text{if } F(Y_{\text{new}}) > F(Y_{\text{best}}) \\ r^0, & \text{if } F(Y_{\text{new}}) > F(Y_{\text{late}}) \\ r^-, & \text{otherwise} \end{cases} \quad (5)$$

where $P(Y_{\text{late}})$ is a historical solution recorded during optimization that follows a first-in-first-out rule and is dynamically updated as iterations proceed. This mechanism allows

Table 3

Experimental results of different solvers across different problems.

Problem	CPLEX			OR-Tools			LMeHA	
	GAP(%)	Time(s)	Fail.	GAP(%)	Time(s)	Fail.	GAP(%)	Time(s)
AP	0	322.16	4	0	184.23	4	2.20	7.23
BPP	3.40	1047.85	0	0	354.51	0	0.60	2.58
GCP	3.17	587.99	0	0	225.95	0	0	12.44
JSP	0.57	100.62	0	0	19.57	0	5.64	6.72
KP	0	0.396	0	0	0.394	0	6.20	2.45
NQP	0	2.62	0	0	5.24	0	0	31.23
OPTW	28.79	516.12	0	0.31	513.82	0	4.11	5.14
FO	5.34	453.41	4	3.34	341.21	4	2.31	30.42
TSP	92.71	692.50	2	0	541.17	0	8.68	99.60
CVRPTW	46.99	903.03	3	18.10	600.08	1	0	17.42
LSSP	56.64	803.42	0	38.03	606.53	0	0	32.67
NDP	–	–	5	53.05	600.65	0	0	119.42
SASP	–	–	5	–	–	5	0	10.00
EOSSP	–	–	5	–	–	5	0	58.17
HRSP	–	–	5	–	–	5	0	18.83
MPP	–	–	5	–	–	5	0	15.64
NSSP	–	–	5	–	–	5	0	34.74
AGAP	–	–	5	–	–	5	0	112.83

the acceptance of inferior solutions within a certain range: when a new solution does not improve the current best but outperforms the historical threshold, a small positive reward is still granted.

The temporal difference method is adopted for online Q-value updates, with the update rule given in Eq. (6) and Eq. (7):

$$\delta = R(s, a) + \gamma \max_{a'} Q(s', a') - Q(s, a) \quad (6)$$

$$Q(s, a) \leftarrow Q(s, a) + \alpha \delta \quad (7)$$

where α is the learning rate and γ is the discount factor. Through immediate reward feedback and dynamic state transitions, the Q-table accumulates effective action experience across iterations, thereby improving the precision of operator selection.

3.6. Experimental Setup and Datasets

4. Experimental Evaluation

To validate the effectiveness of the proposed generalized modeling method and algorithm, computational experiments are conducted on multiple classical and engineering COPs. The instance names are listed in Table 2, and all instances and solving code are open-sourced on GitHub¹ and Gitee². The classical COP instances are drawn from OR-Library [48]. The engineering COP instances are derived from real-world scenarios: the EOSSP and SASP instances originate from the 4th Tianzhi Cup AI Challenge [31, 49]; the NDP instances are sourced from infrastructure network node graphs [50]; the HRSP instances are constructed based

on organizational personnel structure and mobility networks [10]; and the LSSP data are based on actual space launch scheduling records [51].

Comparative experiments are conducted against CPLEX, OR-Tools, and five metaheuristic algorithms implemented within NOPT: Genetic Algorithm (GA)[30], Hill Climbing (HC)[22], Iterated Local Search (ILS)[26], Late Acceptance (LA)[23], and Simulated Annealing with Tabu Strategy (SA/T)[24]. To ensure experimental fairness, all algorithms use identical operator pool configurations when solving the same COP. For neighborhood search algorithms, the maximum number of iterations is set to 5,000,000 (outer loop count adjusted accordingly for algorithms with inner loops), the tabu tenure and late acceptance list size are set to 5, and the maximum running time is 10 minutes. For evolutionary algorithms, the maximum number of iterations is 500, the population size is 100, the crossover probability is 0.7, and the mutation probability is 0.2. Termination is triggered when the cumulative number of non-improving iterations exceeds 20% of the maximum. The experimental environment consists of Java 1.8.0 on an Intel Core i7-12900H CPU @ 2.40 GHz with 8 GB RAM, and all results are averaged over 10 independent runs.

4.1. Comparison with Commercial Solvers

Table 3 presents the performance of three types of solvers across different COPs, including GAP, solving time, and failure count. GAP is calculated relative to the best result obtained for each problem, with 0% indicating the best performance and other values representing the relative deviation from the optimum. “Fail.” denotes the number of instances on which the solver failed to produce a solution. “–” indicates that the solver cannot handle the corresponding

¹<https://github.com/duyonghao15/NOPT>

²<https://gitee.com/duyonghao15/NOPT>

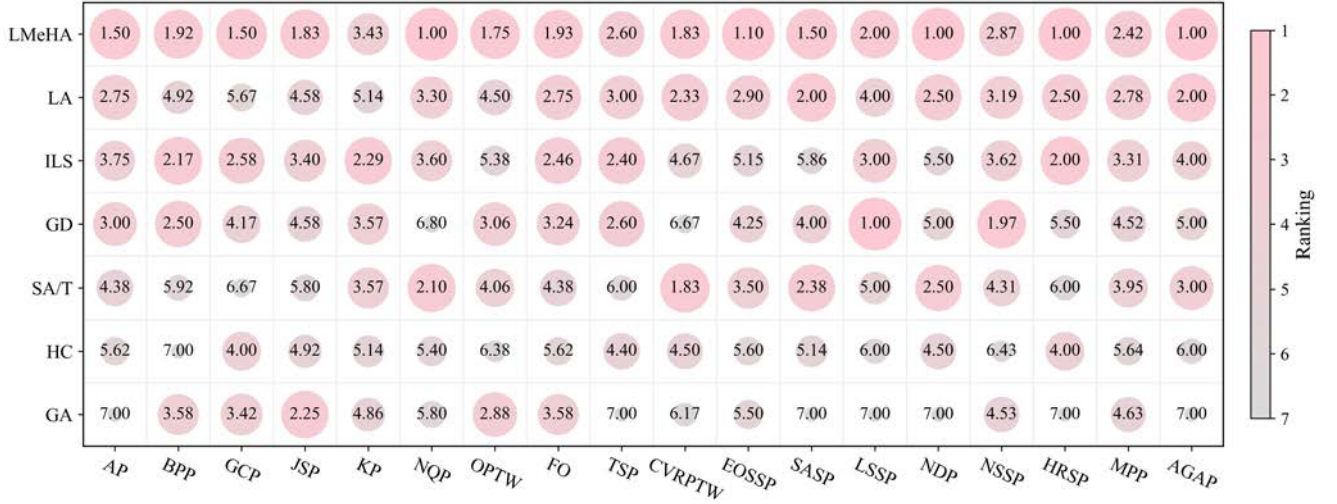


Figure 5: Heatmap of metaheuristic algorithm rankings in different problems.

problem type due to limited constraint expressiveness or large problem scales.

On classical COPs, OR-Tools achieves the best overall solving performance, while CPLEX holds advantages on certain problems. LMeHA shows a clear computational time advantage but a larger optimality gap on problems where efficient exact algorithms exist, such as KP with pseudo-polynomial-time methods and NQP with structured constraint pruning. On FO, both CPLEX and OR-Tools cannot handle high-degree polynomials due to variable scale and value precision limitations, resulting in failures on several instances. As problem complexity increases, LMeHA’s computational time advantage becomes more pronounced and its solution quality gap relative to exact solvers narrows considerably. On TSP and OPTW, CPLEX’s solving performance degrades severely, whereas OR-Tools achieves superior results through its dedicated routing optimization module. On CVRPTW, however, LMeHA outperforms both CPLEX and OR-Tools in solution quality and computational time.

On complex COPs, exact solvers are constrained by their limited constraint expressiveness and problem scale handling. For NDP, CPLEX fails to complete constraint processing and analysis within the specified time due to the large instance scale. OR-Tools can generate initial solutions through heuristic algorithms, but subsequent optimization remains limited. LSSP is essentially a flexible job shop scheduling problem, yet it involves dynamic time windows, coupled resource constraints, and soft benefit requirements. Although exact solvers can formulate the problem model, their optimization performance is unsatisfactory. For SASP, EOSSP, HRSP, and AGAP, the modeling process involves nonlinear, non-convex constraints or objective functions—such as position evaluation, area coverage rate computation, and observation interval calculation—that cannot be directly modeled. Combined with the large problem scale, these

factors render exact solvers incapable of solving these problems, further demonstrating NOPT’s advantages in problem coverage and complex constraint handling.

4.2. Comparison with Metaheuristic Algorithms

Fig. 5 presents a heatmap of rankings for metaheuristic algorithms across various problem types, where each cell value denotes the average ranking of an algorithm on the corresponding problem scenarios. Because different COPs exhibit distinct solution-space characteristics and constraint structures, LMeHA, which integrates the exploration capability with the exploitation capability, achieves optimal or near-optimal rankings on the majority of problems. Among the baselines, LA, ILS, and GD yield competitive results on certain problems without significant mutual differences. HC, owing to its relatively simple iterative mechanism, is prone to local optima and ranks comparatively low. GA exhibits notable problem dependency. On JSP and OP, where neighborhood search algorithms are susceptible to local optima, evolutionary algorithms benefit from population diversity and outperform neighborhood search methods; however, GA performs poorly on large-scale, complex-constrained problems.

To evaluate the overall performance across all test problems, we applied the Friedman test followed by the Nemenyi post-hoc test based on the ranking results. As presented in Table 4, the Friedman test yields a statistic of 52.75 with a p -value less than 0.001. This strongly rejects the null hypothesis, confirming that the performance differences among the compared algorithms are statistically significant. To identify exactly which algorithms differ, the Nemenyi post-hoc test is conducted. The critical difference (CD) is 2.124 at the $\alpha = 0.05$ significance level and 1.938 at $\alpha = 0.10$; a statistically significant difference exists between two algorithms if their average ranking difference exceeds the CD value. Based on the average rankings, LMeHA ranks first. At the 0.05 significance level, LMeHA demonstrates

Table 4

Average rankings and statistical significance test results for each algorithm.

Metric / Algorithm	LMeHA	LA	ILS	GD	SA/T	HC	GA
Average Rank	1.79	3.37	3.62	3.97	4.19	5.34	5.35
Statistics Value	Algorithms k 7	Problems N 18	$q_{\alpha=0.05}$ 2.949	$CD_{\alpha=0.05}$ 2.124	$CD_{\alpha=0.10}$ 1.938	Friedman stat. 52.75	p -value <0.001

Table 5

Optimization results of different algorithms on complex scenarios.

Scenario	Problem	Instance Scale	LMeHA	LA	ILS	GD	SA/T	HC	GA
1	NDP	2591	573	567	417	559	565	534	1174(-1)
2		33440	1559	1372	1232	1117	1443	1276	2786(-1)
3	OPTW	101	990	987	970	998	998	938	732
4		101	1080	1075	1063	1072	1077	1007	663
5		101	707	586	583	592	589	553	716
6		101	852	785	760	775	788	680	845
7	KP	28	139449	126900	134896	122109	120895	121052	139598
8		28	92108	70245	81076	77514	63437	73870	92288
9		50	6080	6045	6160	6134	6159	6094	5477
10		60	6872	6945	7427	7236	7332	6933	5344
11		70	9022	9148	9270	9212	9179	9004	7655
12	EOSSP	83	53	53	53	53	53	53	53
13		659	698	695	665	691	696	651	667
14		1711	1637	1629	1500	1624	1628	1468	1386
15		3042	2661	2658	2384	2652	2655	2343	2061
16		5211	3895	3883	3446	3872	3871	3425	2733
17	SASP	475	312	312	288	309	311	294	322(-1)
18		439	294	296	277	294	294	282	302(-1)
19		456	305	305	286	302	304	285	310(-1)
20		460	313	311	292	308	312	296	318(-1)
21		447	300	299	284	297	300	285	305(-1)
22		443	290	291	271	288	290	275	299(-1)
23		439	293	293	275	291	292	278	297(-1)

significant advantages over GD, SA/T, HC, and GA, as the rank differences between LMeHA and these algorithms all exceed the CD of 2.124. LA and ILS rank closer to LMeHA than the other baselines. Specifically, their rank differences from LMeHA are 1.58 and 1.83, respectively, which are smaller than the CD of 1.938 at the 0.10 level. Thus, although LMeHA shows a clear numerical lead over LA and ILS, this advantage is not statistically significant. Nevertheless, LMeHA remains the only algorithm that statistically outperforms the majority of the baselines.

Table 5 summarizes the optimization results to analyze the effects of problem complexity and scale on algorithmic performance. Note that the value of -1 in the GA denotes infeasible solutions, where the algorithm violates constraints despite achieving superficially high objective values. A prominent observation is GA's deficiency in constraint handling; it yields infeasible solutions across all NDP and SASP instances. In such tightly constrained and topologically coupled spaces, GA's global crossover operators

struggle to preserve feasibility. In contrast, LMeHA consistently secures the best feasible solutions in most scenarios, demonstrating excellent constraint-aware search capabilities. Crucially, as the scale of decision instances increases, the performance gap between LMeHA and the comparative algorithms widens significantly, confirming its robust scalability.

Regarding broader scalability, GA exhibits a clear performance deterioration as instance scale grows. In KP, GA outperforms baselines only on the smallest scale; as the scale increases, ILS takes the lead. Similarly, in EOSSP, while algorithms perform comparably on the smallest instance, LMeHA increasingly dominates as the scale expands, whereas GA suffers the most severe performance decline. For OPTW, where instance scales are identical but landscapes vary, algorithmic competitiveness is highly sensitive to the topological structure. GA excels in specific cases but collapses in others. In contrast, LMeHA maintains consistent top-tier performance across varying difficulty levels. In SASP, where LA demonstrates strong competitiveness,

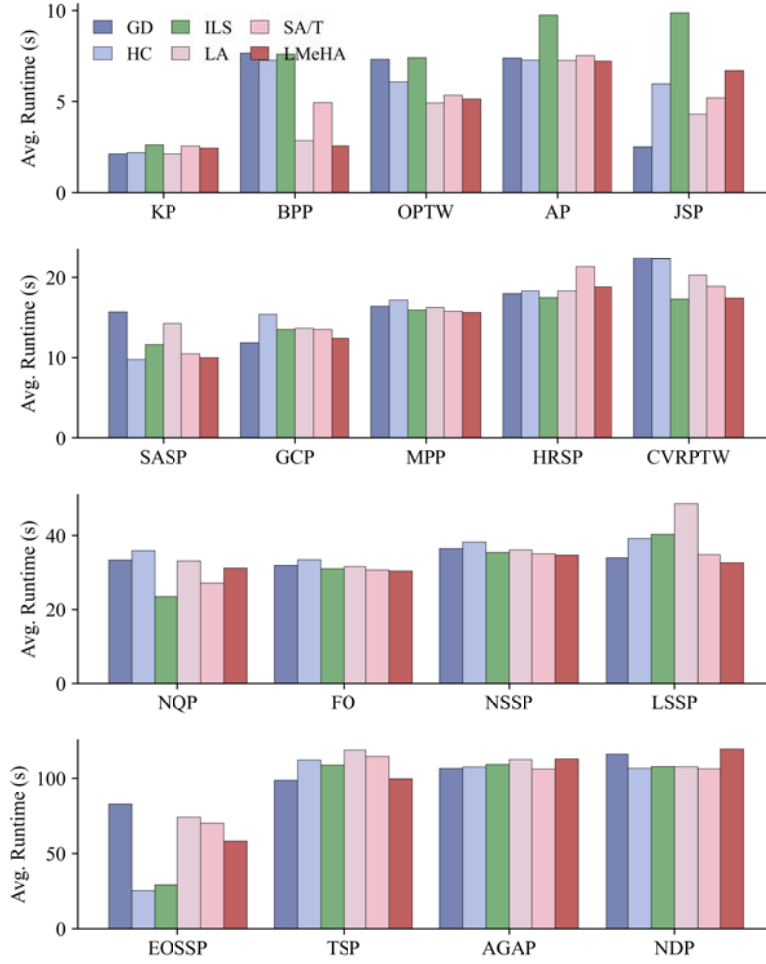


Figure 6: Average computation time of metaheuristic algorithms in different problems.

LMeHA also secures the best feasible results in most scenarios. Overall, while GA relies on deceptive infeasible solutions or limited small-scale advantages, LMeHA exhibits a robust balance between global exploration and constraint-aware exploitation, ensuring consistent scalability and feasibility across complex scenarios.

Table 6 and Figure 6 present the average running times of various metaheuristic algorithms across different problems. GA consistently requires significantly longer computation times than the other algorithms in all scenarios. The remaining algorithms are predominantly neighborhood search-based methods. The last row of Table 5 counts the number of times each algorithm achieves the fastest running time across all problems. LMeHA records the highest count, followed by GD. The other algorithms show no significant differences. This finding indicates that LMeHA achieves faster convergence.

4.3. Ablation and Analysis of LMeHA

To verify the effectiveness of the proposed learning mechanism in LMeHA, ablation experiments are conducted by comparing LMeHA with MeHA where Q-learning is removed. As shown in Table 7 and Fig. 7, LMeHA achieves

a dominant advantage in solution quality measured by Relative Improvement Percentage (RIP), winning on 12 problems and losing on only 4. Specifically, LMeHA performs better on 7 problems. In terms of computational efficiency, LMeHA runs faster on 10 problems, but incurs moderate overhead on the remaining 8 problems, which is mainly due to the additional local search that enhances solution quality. The trade-off analysis further demonstrates the superiority of LMeHA. LMeHA achieves the most desirable outcome on 4 problems and maintains equal solution quality with faster speed on 2 problems. Notably, on 8 problems, LMeHA delivers significantly better solution quality at the cost of slightly increased computation time, which is an acceptable trade-off for most real-world applications. Overall, the results confirm that the learning mechanism provides a favorable balance between solution quality and computational efficiency.

Fig. 8 illustrates the convergence profiles of the algorithms across the tested COPs. On the AP, GCP, TSP, and LSSP, LMeHA exhibits a slower convergence rate during the initial search phase. This early-stage behavior stems from the

Table 6

Average running time (s) of different algorithms across different problems.

Problem / Algorithm	GA	GD	HC	ILS	LA	SA/T	LMeHA
AP	35.88	7.40	7.29	9.75	7.27	7.54	7.23
BPP	6.94	7.67	7.28	7.61	2.86	4.94	2.58
GCP	50.19	11.89	15.42	13.53	13.69	13.53	12.44
JSP	8.56	2.53	5.97	9.87	4.31	5.20	6.72
KP	2.93	2.14	2.21	2.64	2.14	2.57	2.45
NQP	247.73	33.40	35.90	23.53	33.13	27.17	31.23
OPTW	101.83	7.33	6.08	7.42	4.92	5.35	5.14
FO	228.15	31.94	33.46	31.03	31.64	30.72	30.42
TSP	581.33	98.73	112.07	108.77	118.63	114.40	99.73
CVRPTW	252.03	22.39	22.33	17.31	20.28	18.89	17.42
LSSP	507.83	34.00	39.17	40.33	48.67	34.83	32.67
NDP	329.42	116.00	106.58	107.75	107.58	106.25	119.42
SASP	245.95	15.71	9.76	11.67	14.29	10.48	10.00
EOSSP	91.08	82.73	25.22	29.07	73.98	70.13	58.17
HRSP	211.17	18.00	18.33	17.50	18.33	21.33	18.83
MPP	109.48	16.42	17.20	15.95	16.27	15.80	15.64
NSSP	277.92	36.48	38.21	35.43	36.13	35.09	34.74
AGAP	269.33	106.50	107.50	109.17	112.50	106.17	112.83
Best count	0	4	2	3	2	2	6

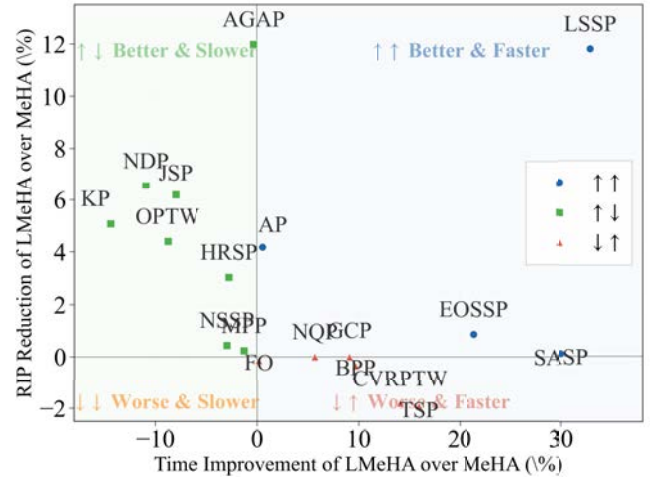
Table 7

Ablation experimental results of LMeHA and MeHA.

Problem	RIP (%)		Time Imp.	Trade-off†
	LMeHA	MeHA		
AP	0.64	4.82	0.57	↑↑
BPP	0.34	0.00	9.71	↓↑
GCP	0.00	0.00	9.13	=↑
JSP	0.00	6.20	-8.06	↑↓
KP	0.35	5.43	-14.44	↑↓
NQP	0.00	0.00	5.73	=↑
OPTW	0.63	5.03	-8.82	↑↓
FO	0.57	0.43	0.21	↓↑
TSP	1.94	0.01	15.93	↓↑
CVRPTW	1.82	0.01	14.11	↓↑
LSSP	0.00	11.80	32.88	↑↑
NDP	0.00	6.56	-11.00	↑↓
SASP	0.07	0.17	30.00	↑↑
EOSSP	0.00	0.86	21.38	↑↑
HRSP	0.00	3.02	-2.73	↑↓
MPP	0.21	0.45	-1.24	↑↓
NSSP	1.21	1.65	-2.91	↑↓
AGAP	0.00	11.98	-0.30	↑↓
Summary	Win: 12 / Lose: 4		Faster: 10	

† ↑: LMeHA is better/faster; ↓: LMeHA is worse/slower;
 =: equal. e.g., ↑↑: Faster & Better; ↑↓: Slower & Better.

inherent properties of its constituent components. Specifically, neighborhood search mechanisms are sensitive to initial solution quality and stochastic perturbations. Simultaneously, the reinforcement learning module requires an exploration period to accumulate effective state-action mappings. During the later iterations, however, LMeHA demonstrates an adaptive shift in its search strategy. The learned policy

**Figure 7:** Time-performance improvement distribution of LmeHA with the learning mechanism in different problem.

begins to exploit the accumulated knowledge, effectively guiding the neighborhood search toward promising regions. Consequently, LMeHA achieves final objective values and hard constraint violation counts that are competitive with or superior to those of the comparison algorithms, indicating consistently high solution quality upon convergence. Regarding constraint satisfaction, the GA struggles with feasibility maintenance on complex-constrained problems such as AGAP, NDP, and SASP. It frequently yields infeasible solutions during the decoding process, which leads to a higher frequency of hard constraint violations. This limitation substantially degrades its convergence speed relative to the other evaluated methods.

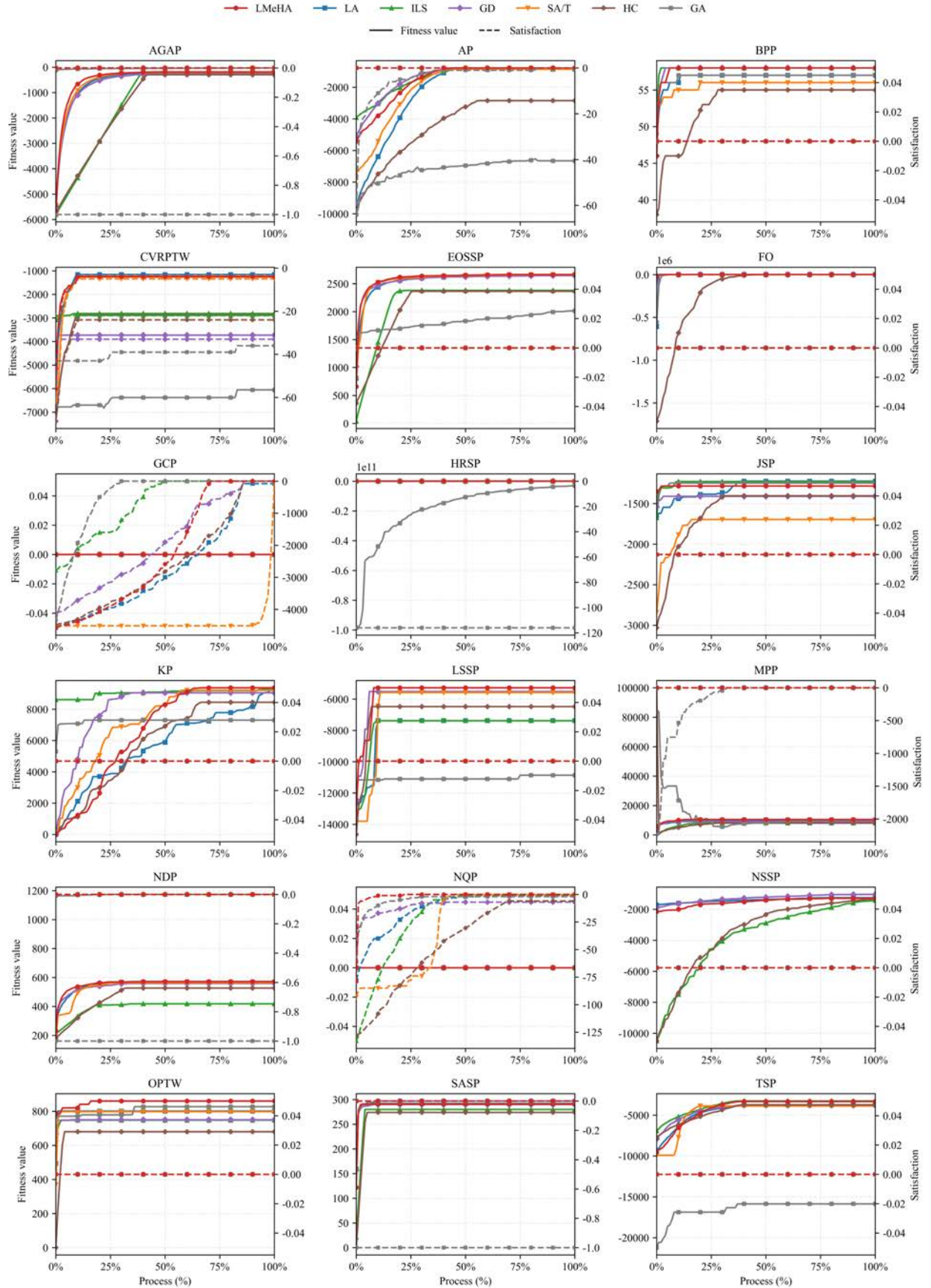


Figure 8: Convergence curves of fitness values and hard constraint violations for different algorithms on COP problems.

5. Conclusions

This paper presents a learning-enhanced metaheuristic framework and algorithm, validated on 10 classical and 8 engineering COPs. A generalized modeling method based on decision instances is proposed, enabling unified description and modeling of diverse COPs. A general-purpose learning-enhanced metaheuristic algorithm framework is designed that accommodates both neighborhood search and evolutionary algorithms, where a reinforcement learning strategy guides the optimization process and substantially enhances the adaptive optimization capability. Comparative experiments on classical and complex industrial COPs demonstrate that the NOPT solver exhibits strong generalizability and solving performance in large-scale, complex-constrained scenarios.

Future work will focus on three directions. First, efficient generalized operators will be designed and integrated to improve NOPT's optimization capability on classical and similar engineering problems. Second, large language models will be incorporated to investigate automated modeling and algorithmic evolution mechanisms for complex problems, enabling more intelligent combinatorial optimization paradigms. Third, generalized hierarchical optimization frameworks will be developed to meet the efficient solving requirements of complex, large-scale industrial problems.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data and code are available at <https://github.com/duyonghao15/NOPT> and <https://gitee.com/duyonghao15/NOPT>.

Acknowledgments

This work was supported by [Fund 1], [Fund 2], and [Fund 3].

References

- [1] Naeimeh Mohseni, Peter L McMahon, and Tim Byrnes. Ising machines as hardware solvers of combinatorial optimization problems. *Nature Reviews Physics*, 4(6):363–379, 2022.
- [2] X. Jiang, J. Li, R. Mallipeddi, et al. Integrating adaptive divide-and-conquer and large language model for scheduling large-scale tasks in electromagnetic satellite systems. *Expert Systems with Applications*, 318:131978, 2026.
- [3] Y. Wu, L. Wang, R. Li, et al. A learning-based dual-population optimization algorithm for hybrid seru system scheduling with assembly. *Swarm and Evolutionary Computation*, 94:101901, 2025.
- [4] F. Yao, Y. Chen, L. Wang, et al. A bilevel evolutionary algorithm for large-scale multiobjective task scheduling in multiagile earth observation satellite systems. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 54(6):3512–3524, 2024.
- [5] Z. Chen, Z. Luo, X. Jin, et al. Multi-UAV path planning problem with biased sampling, candidate evaluation, and path reconfiguration in complex environment with threats. *Expert Systems with Applications*, 291:128558, 2025.
- [6] Kai Meng, Zonghong Jiang, Bin Xin, Fang Deng, and Chen Chen. An adaptive memetic algorithm for multi-uav cooperative task assignment under complex constraints. *Swarm and Evolutionary Computation*, 105:102395, 2026.
- [7] S. Tkatek, O. Bahti, Y. Lmzouari, et al. Artificial intelligence for improving the optimization of NP-hard problems: A review. *International Journal of Advanced Trends in Computer Science and Applications*, 9(5), 2020.
- [8] Y. Du, T. Wang, B. Xin, et al. A data-driven parallel scheduling approach for multiple agile earth observation satellites. *IEEE Transactions on Evolutionary Computation*, 24(4):679–693, 2020.
- [9] Y. Du, L. Li, S. Xu, et al. Evolutionary optimization for satellite constellation task scheduling based on intelligent optimization engine. *Journal of Electronics & Information Technology*, 47(6):1645–1657, 2025. in Chinese.
- [10] J. Huang, J. Li, Y. Du, et al. Research of a multi-level organization human resource network optimization model and an improved late acceptance hill climbing algorithm. *Mathematics*, 11(23):4813, 2023.
- [11] Y. Du, B. Zhang, J. Wu, et al. Survey on intelligent methods for large-scale remote sensing satellite scheduling. *Journal of Electronics & Information Technology*, 47(12):5033–5047, 2025. in Chinese.
- [12] R. Wang, Z. Zhou, K. Li, et al. Learning to branch in combinatorial optimization with graph pointer networks. *IEEE/CAA Journal of Automatica Sinica*, 11(1):157–169, 2024.
- [13] R. Kuroiwa and J. C. Beck. Domain-independent dynamic programming. *Artificial Intelligence*, 354:104506, 2026.
- [14] G. Peng, L. Xing, F. Ma, et al. A learning-augmented dynamic programming approach for orienteering problem with time windows. *Advances in Neural Information Processing Systems*, 38:3163–3188, 2026.
- [15] Xiaolu Liu, Gilbert Laporte, Yingwu Chen, and Renjie He. An adaptive large neighborhood search metaheuristic for agile satellite scheduling with time-dependent transition time. *Computers & Operations Research*, 86:41–53, 2017.
- [16] O. E. Turgut, M. S. Turgut, and E. Kirtepe. A systematic review of the emerging metaheuristic algorithms on solving complex optimization problems. *Neural Computing and Applications*, 35(19):14275–14378, 2023.
- [17] Zuoxun Wang, Changkun Guo, Jinxue Sui, and Chuanyu Cui. Optimization of wireless sensor network deployment based on desert golden mole optimization algorithm. *Intelligence & Robotics*, 5(1):1–18, 2025.
- [18] Guohua Wu, Zhiqing Xiang, Yalin Wang, Yi Gu, and Witold Pedrycz. Improved adaptive large neighborhood search algorithm based on the two-stage framework for scheduling multiple super-agile satellites. *IEEE Transactions on Aerospace and Electronic Systems*, 60(5):7185–7200, 2024.
- [19] F. Wang, J. Chen, Y. Du, et al. LLM-assisted adaptive large neighborhood search for agile earth observation satellite scheduling. 2025. Preprint.
- [20] Shiqi Liu, Xingmin Wang, Yang Wu, Qian Li, Jiuxiang Yan, and Eugene Levin. Path planning method for usvs based on improved dwa and colregs. *Intelligence & Robotics*, 4(4):385–405, 2024.
- [21] Zhongqiang Ma, Guohua Wu, Ponnuthurai Nagarathnam Suganthan, Aijuan Song, and Qizhang Luo. Performance assessment and exhaustive listing of 500+ nature-inspired metaheuristic algorithms. *Swarm and Evolutionary Computation*, 77:101248, 2023.
- [22] Sathiyaraj Chinnaamy, M Ramachandran, M Amudha, and Kurinji-malar Ramu. A review on hill climbing optimization methodology. *Recent trends in management and commerce*, 3(1):1–7, 2022.
- [23] Edmund K Burke and Yuri Bykov. The late acceptance hill-climbing heuristic. *European Journal of Operational Research*, 258(1):70–78, 2017.
- [24] B. Chopard and M. Tomassini. Simulated annealing. In *An introduction to metaheuristics for optimization*, pages 81–106. Springer, 2026.

- [25] J. Xie, X. Li, L. Gao, et al. A hybrid genetic tabu search algorithm for distributed flexible job shop scheduling problems. *Journal of Manufacturing Systems*, 71:82–94, 2023.
- [26] H. Duan, X. Li, G. Zhang, et al. Elite-based multi-objective improved iterative local search algorithm for time-dependent vehicle-drone collaborative routing problem with simultaneous pickup and delivery. *Engineering Applications of Artificial Intelligence*, 139:109608, 2025.
- [27] Hadi Moazen, Sajjad Molaei, Leili Farzinvas, and Masoud Sabaei. Pso-elpm: Pso with elite learning, enhanced parameter updating, and exponential mutation operator. *Information Sciences*, 628:70–91, 2023.
- [28] Dynhora-Danheyda Ramírez-Ochoa, Luis Asunción Pérez-Domínguez, Erwin-Adán Martínez-Gómez, and David Luviano-Cruz. Pso, a swarm intelligence-based evolutionary algorithm as a decision-making strategy: A review. *Symmetry*, 14(3):455, 2022.
- [29] X. Ma, X. Li, Q. Zhang, et al. A survey on cooperative co-evolutionary algorithms. *IEEE Transactions on Evolutionary Computation*, 23(3):421–441, 2019.
- [30] Bushra Alhijawi and Arafat Awajan. Genetic algorithms: Theory, genetic operators, solutions, and applications. *Evolutionary intelligence*, 17(3):1245–1256, 2024.
- [31] L. Li, Y. Du, F. Yao, et al. Learning memetic algorithm based on variable population and neighborhood for multi-complex target scheduling of large-scale imaging satellites. *Swarm and Evolutionary Computation*, 92:101789, 2025.
- [32] Shang Xiang, Ling Wang, Lining Xing, and Yonghao Du. An effective memetic algorithm for uav routing and orientation under uncertain navigation environments. *Memetic Computing*, 13(2):169–183, 2021.
- [33] R. Hu, Y. F. Huang, X. Wu, et al. Collaborative Q-learning hyper-heuristic evolutionary algorithm for the production and transportation integrated scheduling of silicon electrodes. *Swarm and Evolutionary Computation*, 86:101498, 2024.
- [34] J. Wu, F. Yao, Y. Song, and others. Frequent pattern-based parallel search approach for time-dependent agile earth observation satellite scheduling. *Information Sciences*, 636:118924, 2023.
- [35] Yanjie Song, Luona Wei, Qing Yang, Jian Wu, Lining Xing, and Yingwu Chen. RL-ga: A reinforcement learning-based genetic algorithm for electromagnetic detection satellite scheduling problem. *Swarm and Evolutionary Computation*, 77:101236, 2023.
- [36] M. Chen, Y. Du, K. Tang, et al. Learning to construct a solution for the agile satellite scheduling problem with time-dependent transition times. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 54(10):5949–5963, 2024.
- [37] C. Zhang, W. Song, Z. Cao, et al. Learning to dispatch for job shop scheduling via deep reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 33, pages 1621–1632. Curran Associates, Inc., 2020.
- [38] S. Nickel, C. Steinhardt, H. Schlenker, et al. Decision optimization with ibm ilog cplex optimization studio. *Angewandte Optimierung mit IBM ILOG CPLEX Optimization Studio*; Springer: Berlin/Heidelberg, Germany, 2022.
- [39] J. P. Pedroso. Optimization with gurobi and python. *INESC Porto and Universidade do Porto*, Porto, Portugal, 1, 2011.
- [40] L. Huang, F. Xiao, D. Ge, et al. An overview of mathematical programming solvers: Theory, development, and recent advances. *Engineering Management*, 13:1–16, 2026.
- [41] Suresh Bolusani, Mathieu Besançon, Ksenia Bestuzheva, Antonia Chmiela, João Dionísio, Tim Donkiewicz, Jasper van Doornmalen, Leon Eifler, Mohammed Ghannam, Ambros Gleixner, et al. The scip optimization suite 9.0. *arXiv preprint arXiv:2402.17702*, 2024.
- [42] Mengyuan Zhang, Wotao Yin, Mengchang Wang, Yangbin Shen, Peng Xiang, You Wu, Liang Zhao, Junqiu Pan, Hu Jiang, and KuoLing Huang. Mindopt tuner: Boost the performance of numerical software by automatic parameter tuning. *arXiv preprint arXiv:2307.08085*, 2023.
- [43] L. Perron and F. Didier. Google or-tools-cp-sat. *Google*, 2025.
- [44] Liwen Zhang, Sara Maqrot, Florent Mouysset, and Christophe Bortolaso. Bl. optim: an optaplanner based optimizer towards resolution of large-scale realistic scheduling and routing problems. In *International Conference on Operations Research (OR 2024)*, 2024.
- [45] A. Song and G. Wu. REMS: a unified solution representation, problem modeling and metaheuristic algorithm design for general combinatorial optimization problems. *arXiv preprint arXiv:2505.17108*, 2025.
- [46] M. A. Rahman, R. Sokkalingam, M. Othman, et al. Nature-inspired metaheuristic techniques for combinatorial optimization problems: Overview and recent advances. *Mathematics*, 9(20):2633, 2021.
- [47] J. Clifton and E. Laber. Q-learning: Theory and applications. *Annual Review of Statistics and Its Application*, 7(1):279–301, 2020.
- [48] J. E. Beasley. OR-Library: distributing test problems by electronic mail. *Journal of the Operational Research Society*, 41(11):1069–1072, 1990.
- [49] S. Cao, X. Liu, and L. He. Three-stage hybrid scheduling method based on link adjustment and arc replacement for large-scale satellite ground station resource scheduling considering idleness rate. *Results in Engineering*, 27:105981, 2025.
- [50] R. A. Rossi and N. K. Ahmed. The network data repository with interactive graph analytics and visualization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2015.
- [51] R. Lin, X. Liu, Y. Du, et al. Q-Learning Enhanced Memetic Algorithm for Robust Scheduling of Space Launch Missions. *Journal of System Simulation*, pages 1–13, 2025. in Chinese.

A. Decision Instance of Different Problems in NOPT

The decision instance is the fundamental data structure in NOPT for unified COP modeling. This appendix presents the Java implementation of decision instances for all 18 classical and engineering COPs tested in this paper. For each problem, the decision instance class inherits from `DecisionEntity`, with decision variables annotated by `@DecisionVariable` and their feasible domains stored in the corresponding optional list.

Classical Combinatorial Optimization Problems

Assignment Problem (AP). The decision instance is *Task*, representing a job to be assigned. The decision variable is *Resource* (instantiated object type), indicating which resource is assigned to the task. The variable domain is the set of available resources, and an auxiliary cost map records the assignment cost for evaluation.

```

1 public class Task extends DecisionEntity {
2     @DecisionVariable
3     private Resource resource; // Assigned resource
4     private List<Resource> optionalResourceList = new ArrayList<>(); // Decision variable domain
5     private Map<Resource, Long> resourceCostMap = new HashMap<>(); // Assignment cost per resource
6     // getter & setter
7     ...
8 }

```

Listing 1: Decision instance implementation for AP.

Bin Packing Problem (BPP). The decision instance is *Item*, representing a rectangular item to be packed. It embeds two decision variables: *Packing priority* (positive integer), which determines the placement order within a bin, and *Rotation flag* (Boolean), which indicates whether the item is rotated by 90°. Additional geometric attributes support constraint checking.

```

1 public class Item extends DecisionEntity {
2     private double width; // Item width
3     private double height; // Item height
4     private double x; // Bottom-left X coordinate
5     private double y; // Bottom-left Y coordinate
6     private Bin bin; // Assigned bin
7     @DecisionVariable(sortIn = "bin")
8     private Integer priority; // Decision variable 1: packing priority
9     @DecisionVariable
10    private Boolean rotate; // Decision variable 2: rotation flag
11    private List<Integer> optionalPriorityList = new ArrayList<>(); // Priority domain
12    private List<Boolean> optionalRotateList = new ArrayList<>(); // Rotation domain
13    // getter & setter
14    ...
15 }

```

Listing 2: Decision instance implementation for BPP.

Graph Coloring Problem (GCP). The decision instance is *Node*, representing a vertex in the graph. The decision variable is *Color* (instantiated object type), indicating which color is assigned to the node. The adjacent node list is maintained to check the hard constraint that no two adjacent nodes share the same color.

```

1 public class Node extends DecisionEntity {
2     @DecisionVariable
3     private Color color; // Decision variable: assigned color
4     private List<Node> adjacentNodeList = new ArrayList<>(); // Adjacent nodes (color-conflict check)
5     private List<Color> optionalColorList = new ArrayList<>(); // Decision variable domain
6     // getter & setter
7     ...
8 }

```

Listing 3: Decision instance implementation for GCP.

Job Shop Problem (JSP). The decision instance is *Operation*, representing a processing step of a job. It embeds two decision variables: *Machine* (instantiated object type), specifying which machine processes the operation, and *Operation priority* (positive integer), determining the sequencing order among operations sharing the same machine. Precedence constraints are modeled via the `preOperation` reference.

```

1 public class Operation extends DecisionEntity {
2     private Job job; // Belonging job
3     private Operation preOperation; // Predecessor operation (precedence constraint)
4     private long beginTime; // Start time
5     private long endTime; // End time
6     @DecisionVariable
7     private Machine machine; // Decision variable 1: assigned machine
8     @DecisionVariable
9     private Integer priority; // Decision variable 2: processing priority
10    // getter & setter
11    ...
12 }

```

Listing 4: Decision instance implementation for JSP.

Knapsack Problem (KP). The decision instance is *Item* (named *Object* in code), representing an object with value and multi-dimensional properties. The decision variable is *Knapsack* (instantiated object type, nullable), indicating whether and into which knapsack the object is placed. A null value means the object is not selected. Multi-dimensional property lists support multi-constraint knapsack variants.

```

1 public class Object extends DecisionEntity {
2     @DecisionVariable(nullable = true)
3     private Knapsack knapsack; // Decision variable: assigned knapsack (null = not selected)
4     private List<Knapsack> optionalKnapsackList = new ArrayList<>(); // Decision variable domain
5     private double value; // Item value
6     private List<Double> propertyList = new ArrayList<>(); // Multi-dimensional property list
7     private Map<String, Double> propertyMap = new HashMap<>(); // Multi-dimensional property map
8     // getter & setter
9     ...
10 }

```

Listing 5: Decision instance implementation for KP.

N-Queens Problem (N-QP). The decision instance is *Queen*, representing a queen on the chessboard. The decision variable is *Grid cell* (instantiated object type, named *Square*), indicating which square on the corresponding column the queen occupies. Each queen is restricted to a single column, so the variable domain contains all squares within that column.

```

1 public class Queen extends DecisionEntity {
2     @DecisionVariable
3     private Square square; // Decision variable: assigned grid cell
4     private List<Square> optionalSquareList = new ArrayList<>(); // Decision variable domain (squares in the given column)
5     // getter & setter
6     ...
7 }

```

Listing 6: Decision instance implementation for N-QP.

Orienteering Problem with Time Windows (OPTW). The decision instance is *Vertex*, representing a point of interest. It embeds two decision variables: *Route* (instantiated object type, nullable), specifying which tour visits this vertex, and *Route priority* (positive integer), determining the visiting order within the assigned tour. A null route means the vertex is skipped. Each vertex carries a score, service duration, and time window for constraint evaluation.

```

1 public class Vertex extends DecisionEntity {
2     @DecisionVariable(nullable = true)
3     private Tour tour; // Decision variable 1: assigned route (null = skipped)
4     @DecisionVariable(sortIn = "tour")
5     private Integer priority = 0; // Decision variable 2: visiting priority within the route
6     private List<Tour> optionalTourList = new ArrayList<>(); // Route domain
7     private List<Integer> optionalPriorityList = new ArrayList<>(); // Priority domain
8     private double x; // X coordinate
9     private double y; // Y coordinate
10    private double score; // Collected score
11    private long duration; // Service duration
12    private TimeWindow timeWindow; // Service time window
13    // getter & setter
14    ...
15 }

```

Listing 7: Decision instance implementation for OPTW.

Function Optimization (FO). The decision instance is *Variable* (named *x*), representing a continuous decision variable discretized for optimization. The decision variable is *Discrete value* (floating-point number), which takes values from a discretized domain defined by a minimum, maximum, and precision step.

```

1 public class X extends DecisionEntity {
2     @DecisionVariable
3     private Double value; // Decision variable: discrete value
4     private List<Double> optionalValueList; // Decision variable domain
5     private double minValue; // Minimum value
6     private double maxValue; // Maximum value
7     private double precision = 0.01; // Discretization precision
8     // getter & setter
9     ...
10 }

```

Listing 8: Decision instance implementation for FO.

Traveling Salesman Problem (TSP). The decision instance is *City*, representing a node in the tour. The decision variable is *Priority* (positive integer), which determines the visiting order of all cities. The *sortIn* parameter groups cities under the same salesman for sorting-based decoding. Auxiliary fields include coordinates and cumulative distance for efficient objective evaluation.

```

1 public class City extends DecisionEntity {
2     @DecisionVariable(sortIn = "salesman")
3     private Integer priority; // Decision variable: visiting priority
4     private Salesman salesman; // Salesman (single for classical TSP)
5     private List<Integer> optionalPriorityList = new ArrayList<>(); // Priority domain
6     private double x; // X coordinate
7     private double y; // Y coordinate
8     private double totalDistance; // Cumulative tour distance
9     // getter & setter
10    ...
11 }

```

Listing 9: Decision instance implementation for TSP.

Capacitated Vehicle Routing Problem with Time Windows (CVRPTW). The decision instance is *Customer*, representing a service point. It embeds two decision variables: *Vehicle* (instantiated object type), assigning the customer to a specific vehicle, and *Access priority* (positive integer), determining the visiting sequence within that vehicle's route. Each customer also carries demand, service duration, and a time window for capacity and time feasibility checks.

```

1 public class Customer extends DecisionEntity {
2     @DecisionVariable
3     private Vehicle vehicle; // Decision variable 1: assigned vehicle
4     @DecisionVariable(sortIn = "vehicle")
5     private Integer priority = 0; // Decision variable 2: visiting priority within the route
6     private List<Vehicle> optionalVehicleList = new ArrayList<>(); // Vehicle domain
7     private List<Integer> optionalPriorityList = new ArrayList<>(); // Priority domain
8     private double x; // X coordinate
9     private double y; // Y coordinate
10    private double demand; // Customer demand
11    private long duration; // Service duration
12    private TimeWindow timeWindow; // Service time window
13    // getter & setter
14    ...
15 }

```

Listing 10: Decision instance implementation for CVRPTW.

Engineering Combinatorial Optimization Problems

Earth Observation Satellite Scheduling Problem (EOSSP). The decision instance is *Task*, representing an imaging request. It embeds two decision variables: *Imaging time window* (instantiated object type, nullable), selecting a specific visible time window for image acquisition, and *Extend flag* (integer), controlling the strip extension for area targets. Auxiliary structures track related tasks, parent tasks for merging, and decoding orders for complex constraint resolution.

```

1 public class Task extends DecisionEntity {
2     @DecisionVariable(nullable = true)

```

```

3 private ImageWindow imageWindow; // Decision variable 1: imaging time window
4 @DecisionVariable
5 private Integer extend = 0; // Decision variable 2: strip extension index
6 private List<ImageWindow> optionalImageWindowList = new ArrayList<>(); // Imaging window domain
7 private List<Integer> optionalExtendList = new ArrayList<>(); // Extension index domain
8 private List<Integer> optionalOrderList = Arrays.asList(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10);
9 private List<Task> relatingTaskList = new ArrayList<>(); // Related tasks (area-target strips, excluding self)
10 private List<Task> fatherTaskList = new ArrayList<>(); // Parent tasks eligible for merging
11 // getter & setter
12 ...
13 }

```

Listing 11: Decision instance implementation for EOSSP.

Satellite Antenna Scheduling Problem (SASP). The decision instance is *Task*, representing a satellite communication service request. The decision variable is *Time window* (instantiated object type, nullable, named *Range*), selecting an antenna visible arc for the service. Each task records setup, duration, and release times, the associated satellite and day, and a support-type map indicating whether the arc supports TTC, DDT, or both. Potential conflict tasks sharing ground-station arcs are pre-indexed for efficient constraint checking.

```

1 public class Task extends DecisionEntity {
2     @DecisionVariable(nullable = true)
3     private Range range; // Decision variable: assigned antenna time window
4     private List<Range> optionalRangeList; // Time window domain
5     private long setup; // Setup time
6     private long duration; // Service duration
7     private long release; // Release time
8     private long count; // Pass index (per day)
9     private Day day; // Scheduling day
10    private Day satelliteDay; // Satellite day
11    private Satellite satellite; // Assigned satellite
12    private TaskRequest taskRequest; // Original request
13    private List<Constraint> constraintList = new ArrayList<>(); // Domain-specific constraints
14    private Map<Range, String> supportTypeMap = new HashMap<>(); // Arc support type (TTC, DDT, TTC/DDT, TTC+DDT)
15    private List<Task> possibleConflictTasksList = new ArrayList<>(); // Potentially conflicting tasks
16    // getter & setter
17    ...
18 }

```

Listing 12: Decision instance implementation for SASP.

Launch Site Scheduling Problem (LSSP). The decision instance is *Task* (named *Job* in code), representing a launch preparation operation. It embeds two decision variables: *Launch pad* (instantiated object type, named *Shop*), specifying which facility processes the job, and *Priority* (positive integer), determining the sequencing order. Precedence constraints are captured by predecessor and successor job lists, and resource requirements are stored in a resource map. Time-slot indexing supports discrete-time scheduling.

```

1 public class Job extends DecisionEntity {
2     @DecisionVariable
3     private Shop shop; // Decision variable 1: assigned launch pad
4     @DecisionVariable
5     private Integer order = 0; // Decision variable 2: processing order
6
7     private List<Shop> optionalShopList = new ArrayList<>(); // Launch pad domain
8     private List<Integer> optionalOrderList = new ArrayList<>(); // Order domain
9     private Launch launch; // Belonging launch mission
10    private List<Job> lastJobList = new ArrayList<>(); // Predecessor operations
11    private List<Job> nextJobList = new ArrayList<>(); // Successor operations
12    private Map<String, Resource> resourceMap = new HashMap<>(); // Required resources
13    private long duration; // Operation duration (min)
14    private int slotNum; // Number of occupied time slots
15    private Job lastJob; // Immediate predecessor
16    private Job nextJob; // Immediate successor
17    private Timeslot beginTimeslot; // Start time slot
18    private Timeslot endTimeslot; // End time slot
19    // getter & setter
20    ...
21 }

```

Listing 13: Decision instance implementation for LSSP.

Network Disintegration Problem (NDP). NDP employs two decision instances: *Node* and *Edge* (named *Link*). Both share the same decision variable *Disintegration flag* (Boolean type), indicating whether the element is removed from the network. The *Node* class maintains its incident edge list for topological constraint evaluation, while the *Link* class records its endpoints and weight.

```

1 public class Node extends DecisionEntity {
2     private List<Link> linkList = new ArrayList<>();           // Incident edges from this node
3     @DecisionVariable
4     private Boolean disintegrate = false;                     // Decision variable: disintegration flag
5     private List<Boolean> optionalDisintegrateList = new ArrayList<>(); // Decision variable domain
6     // getter & setter
7     ...
8 }

```

Listing 14: Decision instance implementation for NDP (Node).

```

1 public class Link extends DecisionEntity {
2     private Node startNode;                                   // Start node
3     private Node endNode;                                     // End node
4     private double weight = 1;                                // Edge weight
5     @DecisionVariable
6     private Boolean disintegrate = false;                     // Decision variable: disintegration flag
7     private List<Boolean> optionalDisintegrateList = new ArrayList<>(); // Decision variable domain
8     // getter & setter
9     ...
10 }

```

Listing 15: Decision instance implementation for NDP (Link).

Human Resource Scheduling Problem (HRSP). The decision instance is *Person* (named *Link* in code, representing a personnel transfer relationship). The decision variable is *Transferable position* (long integer type, named *number*), indicating the number of people transferred along a directed edge from one organizational unit to another. The variable domain ranges from 0 to the transfer upper bound.

```

1 public class Link extends DecisionEntity {
2     protected Node fromNode;                                   // Source organizational unit
3     protected Node toNode;                                     // Target organizational unit
4     protected long priority;                                    // Transfer priority
5     @DecisionVariable
6     protected Long number;                                     // Decision variable: number of transferred personnel
7     protected List<Long> optionalNumberList;                  // Decision variable domain (0 -- upper bound)
8     // getter & setter
9     ...
10 }

```

Listing 16: Decision instance implementation for HRSP.

Navigation Satellite Scheduling Problem (NSSP). The decision instance is *Task*, representing a link establishment opportunity in a specific time slot. The decision variable is *Link establishment flag* (Boolean/integer type, named *active*), indicating whether the inter-satellite link is activated during this slot. Each task is associated with a specific time slot and link.

```

1 public class Task extends DecisionEntity {
2     private Timeslot timeslot;                                 // Time slot
3     private Link link;                                         // Inter-satellite link
4     @DecisionVariable
5     private Integer active;                                    // Decision variable: link activation flag (0/1)
6     private List<Integer> optionalActiveList;                  // Decision variable domain
7     // getter & setter
8     ...
9 }

```

Listing 17: Decision instance implementation for NSSP.

Multi-platform Path Planning Problem (MPP). The decision instance is *Platform* (named *visit* in code), representing a platform–target assignment pair. The decision variable is *Target point* (instantiated object type, named *Node*), specifying which target node the platform visits. Each visit instance is tied to a specific platform and selects from the set of candidate target nodes.

```

1 public class Visit extends DecisionEntity {
2     private Platform platform;                // Belonging platform
3     @DecisionVariable
4     private Node node;                        // Decision variable: visited target node
5     private List<Node> optionalNodeList;      // Decision variable domain
6     // getter & setter
7     ...
8 }

```

Listing 18: Decision instance implementation for MPP.

Airport Gate Assignment Problem (AGAP). The decision instance is *Task* (named *Plane*), representing an arriving/departing flight. The decision variable is *Gate* (instantiated object type, nullable, named *Position*), specifying the assigned boarding gate. A null value indicates the flight is unassigned. Flight properties (arrival/departure times, international flag, airline) support conflict detection and constraint evaluation. Potentially conflicting flights with overlapping time intervals are pre-computed for efficient feasibility checking.

```

1 public class Plane extends DecisionEntity {
2     private String company;                // Airline code
3     private long inTime;                   // Arrival time (seconds)
4     private long outTime;                  // Departure time (seconds)
5     private Date inDate;                   // Arrival time (date format)
6     private Date outDate;                  // Departure time (date format)
7     private String task;                   // Flight mission
8     private boolean international;         // International flight flag
9     private List<Plane> possibleConflictPlaneList = new ArrayList<>(); // Flights with overlapping time intervals
10    @DecisionVariable(nullable = true)
11    private Position position;              // Decision variable: assigned gate (null = unassigned)
12    // getter & setter
13    ...
14 }

```

Listing 19: Decision instance implementation for AGAP.